

AI reference

Tools: GPT, Gemini

Location: Endings.ts 、CamaraFollow.ts 、LeaderboardUI.ts

Endings.ts

MyPrompt: 我把存檔註解掉可以觸摸過關物件進 ending，沒註解掉就會卡一下然後不進

AI's Response: 當物理引擎在執行 onBeginContact（計算碰撞的瞬間）時，內部狀態是鎖定的。如果在此刻同時發送 Firebase 網路請求，且「下一行」立刻呼叫 loadScene 切換場景（這會銷毀當前所有物件），大腦還來不及把資料送出就被銷毀，且打斷了物理碰撞的結算，最終導致引擎當機卡死。建議加入防重複觸發機制，並利用 scheduleOnce 延遲切換場景。

Refinement & Explanation:

加入 isFinished 變數防止玩家在終點擠壓造成重複觸發，並且在呼叫存檔後，給引擎 0.5 秒的緩衝時間，讓網路請求有時間出去，也讓物理引擎有時間解除鎖定。

改前:

```
onBeginContact(contact, selfCollider, otherCollider) {
    if (otherCollider.node.name === "Player") {
        // 沒有防呆，如果玩家一直推物件會重複觸發

        // 呼叫存檔（需要零點幾秒的網路延遲）
        if (FirebaseManager.instance) {
            FirebaseManager.instance.saveProgress();
        }

        // 下一行立刻毀滅當前場景，導致資料發不出去且物理引擎崩潰
        cc.director.loadScene(this.passSceneName);
    }
}
```

改後:

```

private isFinished: boolean = false; // 新增鎖定變數

onBeginContact(contact, selfCollider, otherCollider) {
    // 防重複觸發：如果已經過關了，就忽略後續的碰撞
    if (this.isFinished) return;

    if (otherCollider.node.name === "Player") {
        this.isFinished = true; // 鎖定狀態

        // 1. 呼叫存檔
        if (FirebaseManager.instance) {
            FirebaseManager.instance.saveProgress();
        }

        // 2. 延遲 0.5 秒後再切換場景（避開物理碰撞鎖定，並等待網路請求）
        this.scheduleOnce(() => {
            cc.director.loadScene(this.passSceneName);
        }, 0.5);
    }
}

```

CamaraFollow.ts

My Prompt: CAMARA 平滑怎麼做

AI's Response: 建議使用「線性插值」（Linear Interpolation，也就是 Cocos 內建的 `cc.misc.lerp` 函數），讓攝影機每一幀都只往玩家的新位置移動一小段比例的距離，藉此創造出「稍微延遲才跟上」的平滑視覺效果。

Refinement & Explanation(修改與邏輯說明): 一開始相機的 X 座標直接設定成玩家的 X 座標，瑪利歐一加速或停下來，畫面就會很死板地跟著急停。參考 AI 提供的 `lerp` 寫法後，加上了時間差 `dt` 與跟隨速度來控制平滑的程度。

改前:

```

update(dt: number) {
    if (this.playerNode) {
        // 直接將相機座標等同於玩家座標，缺乏緩衝機制
        this.node.x = this.playerNode.x;
    }
}

```

改後:

```

@property
followSpeed: number = 5; // 控制平滑跟隨的速度係數

update(dt: number) {
    if (this.playerNode) {
        // 獲取玩家當前的 X 座標作為目標參數
        let targetX = this.playerNode.x;

        // 使用 cc.misc.lerp (線性插值) 計算平滑過渡的座標
        // 參數配置：(當前相機位置, 目標玩家位置, 幀時間差 * 速度係數)
        let currentX = cc.misc.lerp(this.node.x, targetX, dt * this.followSpeed);

        // 更新相機位置 (維持 Y 軸固定, 防止跳躍時畫面抖動)
        this.node.x = currentX;
    }
}
}

```

LeaderboardUI.ts

My Prompt: 我可以越過 leaderboard 點到輸入框 + leaderboard 沒顯示東西

AI's Response: 此現象源於 Cocos Creator 的事件傳遞 (Event Propagation) 機制與不當的資料獲取時機。針對點擊穿透問題, 建議在 UI 父節點掛載 BlockInputEvents 元件以攔截底層事件。針對排行榜資料未顯示, AI 建議利用節點的 onEnable 生命週期回呼函式 (Lifecycle Callback), 確保面板每次被啟動時, 皆能自動觸發非同步的 Firebase 資料庫查詢, 以避免渲染空資料或舊資料。

Refinement & Explanation: 原先在實作排行榜面板時, 僅透過 active = true 來控制顯示, 造成玩家點擊排行榜區域時, 事件會穿透至底層的帳號輸入框、資料讀取與 UI 顯示未同步, 導致面板開啟時常呈現空白。參考 AI 的建議後, 我改為實作 LeaderboardUI。為該面板掛載 BlockInputEvents 元件以隔離不同 UI 層級間的互動、並利用了 onEnable 確保每次開啟面板時能自動觸發並同步最新資料。

改前:

```

// 舊版作法：將讀取資料的邏輯綁定於外部按鈕，容易導致資料與 UI 顯示不同步
public showLeaderboard() {
    this.node.active = true;
    // 缺乏讀取狀態提示，且未處理事件穿透問題
}

```

改後:

```

@ccclass
export default class LeaderboardUI extends cc.Component {
  @property(cc.Label)
  displayLabel: cc.Label = null;

  // 當節點的 active 被設為 true 時，引擎會自動觸發此生命週期函式
  onEnable() {
    if (this.displayLabel) {
      this.displayLabel.string = "Loading..."; // 提供即時的介面回饋
    }
    this.fetchLeaderboardData(); // 確保每次開啟皆重新獲取最新榜單
  }

  // 封裝面板開啟邏輯，供主選單的排行榜按鈕呼叫
  public openPanel() {
    this.node.active = true;
  }

  // 封裝面板關閉邏輯，供面板上的關閉按鈕呼叫
  public closePanel() {
    this.node.active = false;
  }

  private fetchLeaderboardData() {
    // ... 向 Firebase RTDB 發起非同步請求並更新 displayLabel 的邏輯 ...
  }
}

```